Now how to do these test things with LLM or VLM or imagejev ?Provide me plan first then after approval execution. I have foundry local installed in my windows laptop.

Yes—your **Foundry Local** setup can support AI-assisted regression testing in three useful ways:

- **LLM:** Generate test cases, review failures, summarize logs, and create test data.
- **VLM (vision-language model):** Inspect screenshots and judge whether a UI looks correct.
- **Image generation:** Useful for creating mockups, sample images, or visual test fixtures—not for deciding whether production UI passes. Foundry Local's catalog focuses on LLMs, Whisper, and now VLMs; image generation is generally provided through Windows AI APIs on compatible Copilot+ PCs rather than Foundry Local. [1] [2] [3]

Proposed plan

Phase 1: Verify your Foundry Local setup

1. Confirm Foundry Local is installed and working from PowerShell.
2. List available models and identify:
   - A small text LLM, such as a Phi or Qwen model.
   - A VLM, such as Qwen Vision, if available in your installed version/catalog. Foundry Local 1.1 added Qwen 3.5 Vision for on-device image-and-text reasoning. [3] [4]
3. Start the optional OpenAI-compatible local REST server so Python can call it like an OpenAI API. Foundry Local supports `/v1/chat/completions` and the Responses API. [5] [6] [3]

Deliverable: A working local endpoint and a list of models your laptop can run.

Phase 2: Build the regression-test project

We will create a small Python project with:

```
ai-regression-tests/
├─ tests/
│  ├─ test_web_regression.py
│  └─ test_ai_visual_regression.py
├─ screenshots/
├─ reports/
├─ ai/
```

```
|  ├─ foundry_client.py
|  └─ prompts.py
└─ .github/workflows/playwright.yml
```

We will use:

- **Playwright + Pytest** to drive a website and capture screenshots.
- **Foundry Local LLM** to turn a feature description into structured test cases.
- **Foundry Local VLM** to analyze screenshots for missing elements, broken layouts, or unexpected error messages.
- **Pytest** to decide pass/fail based on clear rules, while AI provides analysis and suggestions. [7] [8] [3]

Phase 3: Use the LLM for test design

You will give the local LLM a prompt such as:

> "Generate 10 regression test cases for a login page. Return JSON with title, steps, test data, expected result, and priority."

The LLM can help create:

- Positive tests: valid login succeeds.
- Negative tests: wrong password shows an error.
- Boundary tests: empty fields, long inputs, special characters.
- Regression tests: password reset, account lockout, and session behavior.
- Test data: emails, passwords, and edge-case strings.

Important rule: the LLM **suggests** tests; your test suite and assertions remain the source of truth. This avoids unstable AI-generated pass/fail decisions. [9] [10]

Phase 4: Use the VLM for visual regression

After Playwright takes a screenshot, send it to the local VLM with a structured prompt:

> "Compare this screenshot with these expected UI rules. Report only factual issues: missing button, broken layout, error message, unreadable text, or wrong page. Return JSON with `status`, `issues`, and `confidence`."

Good VLM use cases:

- Verify a login page contains username, password, and sign-in controls.
- Detect visible error messages after a failed action.
- Check whether a page appears blank, misaligned, or broken.
- Analyze screenshots from different browsers or screen sizes. Foundry Local's VLM is designed for UI screenshot interpretation, diagrams, documents, and visual question answering. [4] [3]

Phase 5: Use image generation safely

Image generation is optional and should be used for:

- Creating placeholder product images or avatars for test data.
- Making UI mockups for planned features.
- Generating visual fixtures for image-upload tests.

It should **not** be the main regression validator, because generated images do not prove that your real application behaves correctly. On Windows, image generation is more commonly available through Windows AI APIs on Copilot+ PCs; Foundry Local is primarily for local LLMs, audio transcription, and now VLMs. [2] [11] [1]

Phase 6: Add AI to CI/CD

In GitHub Actions:

1. Install Python, Playwright, and browser dependencies.
2. Run deterministic Playwright/Pytest regression tests.
3. Capture screenshots only for selected critical pages or failed tests.
4. Run the VLM analysis locally or through a self-hosted runner with Foundry Local installed.
5. Save AI findings as test artifacts and reports.

A cloud GitHub runner normally cannot access your laptop's Foundry Local server, so for the AI part you would either run the workflow on a **self-hosted runner** on your Windows machine or keep AI analysis as a separate local step. [12] [13]

What we will build first

The first execution will be a minimal, practical project:

1. Check Foundry Local models and endpoint.
2. Install Python packages: `pytest`, `pytest-playwright`, `openai`, and the appropriate Foundry Local SDK.
3. Create one Playwright regression test that opens a page and captures a screenshot.
4. Send that screenshot to your local VLM.
5. Ask the VLM to return structured JSON:

```
{
  "status": "pass_or_fail",
  "issues": [],
  "summary": ""
}
```

6. Print a simple human-readable report.

Before execution

Please confirm these choices:

1. **Use Playwright + Python + Pytest?** Recommended: yes.
2. **Test target:** A public demo site, or one of your own local web apps?
3. **AI role:** LLM for test-case generation, VLM for screenshot checking, or both?
4. **Foundry Local model:** Should I first inspect your installed models and recommend one based on your hardware?
5. **Execution scope:** Start with a local proof of concept first, then add GitHub Actions later?

Reply "**Approved**" and tell me your preferred test target—or simply say "**Use the default plan**"—and I'll begin with Phase 1: checking your Foundry Local installation and available models.

⁂

1. https://learn.microsoft.com/en-in/azure/foundry-local/what-is-foundry-local
2. https://learn.microsoft.com/en-us/windows/ai/overview
3. https://devblogs.microsoft.com/foundry/foundry-local-v1-1/
4. https://devblogs.microsoft.com/foundry/whats-new-in-microsoft-foundry-may-2026/
5. http://www.learn.microsoft.com/en-in/azure/foundry-local/how-to/how-to-integrate-with-inference-sdks
6. https://learn.microsoft.com/en-gb/azure/ai-foundry/foundry-local/reference/reference-rest
7. https://playwright.dev/python/docs/intro
8. https://www.learn.microsoft.com/en-us/azure/ai-foundry/foundry-local/how-to/how-to-integrate-with-inference-sdks
9. https://keploy.io/blog/community/automated-regression-testing
10. https://www.devopssupport.in/blog/the-complete-guide-to-automated-testing-in-ci-cd-pipelines/
11. https://learn.microsoft.com/en-us/windows/ai/
12. https://playwright.dev/python/docs/ci-intro
13. https://playwright.dev/python/docs/ci
14. https://learn.microsoft.com/en-gb/azure/foundry-local/concepts/foundry-local-architecture
15. http://www.learn.microsoft.com/en-us/windows/ai/foundry-local/get-started
16. https://developer.microsoft.com/en-us/windows/local-ai
17. https://learn.microsoft.com/en-us/azure/foundry-local/get-started
18. https://learn.microsoft.com/fil-ph/windows/ai/
19. https://learn.microsoft.com/en-us/azure/azure-arc/agents-tools-foundry-local/overview
20. https://devblogs.microsoft.com/foundry/foundry-local-ga/